EXPERIMENT NO. 1

First Palindromic String in an Array

Programming Language: Java

Name : A Yuvan Charan
Register No. : 192424383

Aim

To write a Java program that finds the first palindromic string in an array of strings, or prints an empty string if none exists.

Question

Given an array of strings words, return the first palindromic string in the array. If there is no such string, return an empty string "". A string is palindromic if it reads the same forward and backward.

Example 1

Input: words = ["abc","car","ada","racecar","cool"]

Output: "ada"

Explanation: The first string that is palindromic is "ada". Note that "racecar" is also palindromic, but it is not the first.

Example 2

Input: words = ["notapalindrome","racecar"]

Output: "racecar"

Explanation: The first and only string that is palindromic is "racecar".

Algorithm

Step 1: Start.

Step 2: Store the input strings in the array words and set foundIndex = -1.

Step 3: For each word, take its length and set left = 0, right = length - 1.

Step 4: While left < right, compare the characters at left and right. If they differ, the word is not a palindrome; otherwise do left++ and right--.

Step 5: If the word is a palindrome, save its index in foundIndex and stop searching the array.

Step 6: If foundIndex is not -1, print words[foundIndex]; otherwise print an empty string.

Step 7: Stop.

Program (Java)

```
public class Main {

    static boolean isPalindrome(String s) {
        int left = 0, right = s.length() - 1;
```

```
        while (left < right) {
            if (s.charAt(left) != s.charAt(right))
                return false;
            left++;
            right--;
        }
        return true;
    }

    static String firstPalindrome(String[] words) {
        for (String w : words) {
            if (isPalindrome(w))
                return w;
        }
        return "";
    }

    public static void main(String[] args) {
        String[] words1 = {"abc", "car", "ada", "racecar", "cool"};
        String[] words2 = {"notapalindrome", "racecar"};

        System.out.println("Output: \"" + firstPalindrome(words1) + "\"");   //
Example 1
        System.out.println("Output: \"" + firstPalindrome(words2) + "\"");   //
Example 2
    }
}
```

Output

```
Output: "ada"
Output: "racecar"

=== Code Execution Successful ===
```

Complexity Analysis

Time Complexity: O(N * M) in the worst case, O(M) in the best case (N = number of words, M = maximum word length).

Space Complexity: O(1)

Result

Thus the program was executed successfully and the output was verified.